

Aritmética modular

Sebastian Mestre

¿Qué es la aritmética modular?

- Sistema de numeración construido sobre el **resto** de la división.
- Dos números son **congruentes módulo** m si su diferencia es un múltiplo de m .

Definición

Dados enteros a, b y $m \geq 1$,

$$a \equiv b \pmod{m} \Leftrightarrow a - b = k \cdot m \text{ para algún } k \in \mathbb{Z}.$$

- En particular, sumar m no cambia la clase:

$$a + m \equiv a \pmod{m}.$$

Propiedades de la congruencia

- La relación $\equiv$ (mód m) es una **relación de equivalencia**:
 - Reflexiva: $a \equiv a$ (mód m).
 - Simétrica: si $a \equiv b$ (mód m), entonces $b \equiv a$ (mód m).
 - Transitiva: si $a \equiv b$ (mód m) y $b \equiv c$ (mód m), entonces $a \equiv c$ (mód m).
- Se comporta bien con las operaciones básicas:

$$\begin{array}{l} a \equiv b \pmod{m} \\ c \equiv d \pmod{m} \end{array} \Rightarrow \begin{cases} a + c \equiv b + d \pmod{m}, \\ a - c \equiv b - d \pmod{m}, \\ a \cdot c \equiv b \cdot d \pmod{m}. \end{cases}$$

Clases de equivalencia

- Para cada entero a , consideramos el conjunto de todos los números congruentes con a módulo m .
- Este conjunto se llama **clase de equivalencia** de a módulo m :

$$[a] = \{ b \in \mathbb{Z} \mid b \equiv a \pmod{m} \}.$$

Ejemplo: $m = 5$

$$[0] = \{ \dots, -5, 0, 5, 10, 15, \dots \}$$

$$[1] = \{ \dots, -4, 1, 6, 11, 16, \dots \}$$

$$[2] = \{ \dots, -3, 2, 7, 12, 17, \dots \}$$

$$[3] = \{ \dots, -2, 3, 8, 13, 18, \dots \}$$

$$[4] = \{ \dots, -1, 4, 9, 14, 19, \dots \}$$

Notar que $[5] = [0]$, es la misma clase.

El conjunto $\mathbb{Z}_m$

- Hay exactamente m clases de equivalencia módulo m :

$$[0], [1], \dots, [m-1].$$

- Al conjunto de estas clases se lo denota típicamente por $\mathbb{Z}_m$:

$$\mathbb{Z}_m = \{[0], [1], \dots, [m-1]\}.$$

- Como la suma, la resta y el producto **respetan** la congruencia, podemos definir operaciones entre clases:

$$[x] + [y] = [x + y], \quad [x] - [y] = [x - y], \quad [x] \cdot [y] = [x \cdot y].$$

- Para todo entero x , existe un único entero $r \in [0, m - 1]$ tal que

$$x \equiv r \pmod{m}.$$

- A este r lo llamamos **representación canónica** de x módulo m .
- Si x es positivo, r es simplemente el resto de dividir x por m .
- Es muy cómodo trabajar siempre con los números en su representación canónica.

Cuidado con % en C++

- En C++, cuando x es negativo, $x \% m$ **no** da la representación canónica (puede ser negativa).
- Podemos corregirlo de forma estándar:

Función de representación canónica

```
1 int const m = 1000000007; // muy importante que sea constante
2
3 int canonica(int x) {
4     return (x % m + m) % m;
5 }
6
```

Operaciones en representación canónica

- Si mantenemos todos los números en $[0, m - 1]$, las operaciones básicas se pueden implementar de forma segura.
- Truco: una versión más rápida que canónica cuando sabemos que el resultado está en $[0, 2m - 1]$.

Implementación

```
1 // igual que canónica en el intervalo  $[0, 2m-1]$ , y m s r pido
2 int nm(int x) {
3     return x - m * (x >= m);
4 }
5
6 // si x e y est n en  $[0, m-1]$ , x + y est en  $[0, 2m-1]$ 
7 int add(int x, int y) {
8     return nm(x + y);
9 }
10
11 // si x e y est n en  $[0, m-1]$ , x + m - y est en  $[0, 2m-1]$ 
12 int sub(int x, int y) {
```


Potenciación modular

- Necesitamos calcular $x^y \bmod m$ muchas veces en programación competitiva.
- Usamos **exponenciación binaria** para obtener costo $O(\log y)$.

Implementación

```
1 // O(log y)
2 int pot(int x, int y) {
3     if (y == 0) return 1;
4     if (y % 2 == 0) return pot(mul(x, x), y / 2);
5     return mul(x, pot(x, y - 1));
6 }
7
```

Inverso multiplicativo

- Si m es primo, podemos usar el **pequeño teorema de Fermat**:

$$x^{m-1} \equiv 1 \pmod{m} \Rightarrow x^{m-2} \equiv x^{-1} \pmod{m}.$$

- Entonces, el inverso multiplicativo se puede calcular en $O(\log m)$.

Implementación

```
1 int inv(int x) {  
2     return pot(x, m - 2);  
3 }  
4
```

Trucos con inversos y factoriales

- A veces queremos todos los inversos $1, 2, \dots, n$ en $O(n)$.
- También es común precomputar factoriales y sus inversos para combinatoria.

Implementaciones típicas

```
1 int inv[n + 1];
2 inv[1] = 1;
3 for (int i = 2; i <= n; i++) {
4     inv[i] = mul(inv[m % i], m - m / i);
5 }
6
7 int fac[n + 1], ifac[n + 1];
8 fac[0] = 1;
9 forn(i, n) fac[i + 1] = mul(fac[i], i + 1); // factoriales
10 ifac[n] = inv(fac[n]);
11 dforn(i, n) ifac[i] = mul(ifac[i + 1], i + 1); // inversos factoriales
12 forr(i, 1, n) inv[i] = mul(ifac[i], fac[i - 1]); // inversos 1..n
13
```

- La aritmética modular trabaja con clases de equivalencia definidas por el resto módulo m .
- Mantener siempre la **representación canónica** simplifica las implementaciones y evita errores.
- Con operaciones básicas (add, sub, mul, pot, inv) podemos resolver muchos problemas de combinatoria y conteo.

Q&A